4 串

章节脉络



串把字符序列作为一个整体处理。学习本章要同时分清三个问题：逻辑上哪些字符属于串，存储时怎样记录字符与长度，匹配时怎样利用已经比较过的信息。本章代码采用王道的顺序串表示：字符从位置 1 开始，`ch[0]`弃用，`length` 另存串长；位置和匹配结果均按这一约定解释。

辨析·空串也有定义

空串长度为 0；空格串的空格都是字符。子串必须连续，子序列可以不连续。

4.1 字符串、子串与基本操作

串是由零个或多个字符组成的有限序列，记为 $S=' a_1a_2\cdots a_n'$；n 为串长。$n=0$ 的是空串。一个或多个空格构成空格串，它含有字符，长度大于 0。串中的字符可以是字母、数字、标点或其他编码字符；具体程序必须约定一个存储单元表示什么，本章算法以一个 `char` 为一个比较单位。

串中任意**连续**字符组成的子序列称为子串，包含该子串的串称为主串。子串在主串中的位置用其第一个字符的位置表示；同一值可能多次出现，例如 `ababa` 中的 `aba` 从位置 1 和 3 开始。模式串是要查找的串，它可能不存在于主串中；不能先把“模式串”当成已找到的子串。

两个串相等，当且仅当长度相同且每个对应字符都相同。串的大小比较采用字典序：从前向后找第一对不同字符，按约定的字符编码次序决定大小；若一方先结束且已比较部分全相同，较短串较小。例如 `wang` 小于 `wangdao`；比较串的大小不是比较串长，也不是将整个串当成十进制数。

串属于线性结构，但串的基本操作通常以一个串或子串作为处理对象，而普通线性表常以单个数据元素为对象。赋值、比较、连接、取子串、替换与定位是不同运算，输入输出和修改对象应写清。

操作	语义与结果	需要检查的条件
`StrAssign(T,chars)`	将字符序列赋给 T	目标容量、字符数
`StrCopy(T,S)`	用 S 的值复制得到 T	独立复制还是共享存储
`StrEmpty(S)`、`StrLength(S)`	判空、取得串长	空串的 length 为 0
`ClearString(S)`	使 S 成为空串	顺序串清空不必释放固定数组
`DestroyString(S)`	销毁并回收动态存储	堆串与定长串的资源行为不同
`Concat(T,S1,S2)`	按先后连接两串得到 T	长度之和能否容纳
`SubString(Sub,S,pos,len)`	取位置 pos 起、长度 len 的子串	起点、长度及末位置合法

操作	语义与结果	需要检查的条件
`StrCompare(S,T)`	返回正、零或负值表示大小关系	字符编码比较规则
`Index(S,T)`	返回 T 在 S 中第一次出现的位置	本章失败返回 0

辨析·先定接口边界

允许空子串时，pos 可以等于 n+1，但 len 只能为 0；越界请求不能读取字符。

4.1.1 子串的起点和长度怎样约束

设 S 长 n，取非空子串时须满足 $1 \le pos \le n$、$len > 0$ 及 $pos + len - 1 \le n$。若允许空子串，可将条件扩展为 $1 \le pos \le n+1$、$len \ge 0$ 及 $pos + len - 1 \le n$；这时 $n+1$ 只允许取长度 0 的串。接口采用哪种边界应明确约定。本章给出的 `SubString` 允许后一种情况；模式匹配的主体分析采用非空模式，空模式在调用入口约定返回位置 1。

4.2 顺序串、堆串与链串

定长顺序存储用连续字符数组保存串，串长由独立成员表示。教材结构以 `MAXLEN` 指定数组单元数。结合本章位置 0 弃用的约定，可直接访问的字符位置是 `1..MAXLEN-1`，所以实现时维持 `0<=length<MAXLEN`。若软件接口要求可保存整整 MAXLEN 个字符，则需要额外分配一个单元，并同步声明容量约定。

```c
#define MAXLEN 255
typedef struct {
    char ch[MAXLEN];
    int length;
} SString;
```

固定数组的容量有限。清空只需令 `length=0`，旧字符即使仍留在数组中，也不再属于有效串。显式记录长度时无需用`'\0'` 作结束标志；若把这种串传给要求 C 字符串的库函数，必须额外转换表示，不能直接把 `ch` 作为同一种接口使用。

教材式顺序串存在不同布局：用 `ch[0]`记录长度、从位置 1 保存字符；另设长度而从位置 0 保存字符；以`'\0'` 结束；另设长度且字符从 1 开始。本章统一采用最后一种。第一种还受长度单元可表示范围限制；结束符方式需要扫描才能求长，并将结束符与有效字符区分。

堆分配存储用指针指向动态申请的连续区，常见定义为：

```c
typedef struct {
    char *ch;
    int length;
} HString;
```

动态分配能按实际需要调整空间，代价是要管理申请、复制、释放和失败情形。赋值一个指针只复制地址，不能自动得到独立字符副本。长度改变且容量不足时，应先申请新空间，成功后搬移并释放旧区，避免申请失败后丢失原串。

辨析·返回符号即可

串比较的正、零、负分别表示大于、相等、小于；正值不要求恰好是 1。

链式存储可以每结点存一个字符，也可以每结点保存多个字符形成串块。若每字符 1 字节而指针 4 字节，单字符结点只计数据与链接时，存储密度为 $1/(1+4)=20\%$；一个结点存 4 字符时可提升至 $4/(4+4)=50\%$。实际平台指针宽度和结构体对齐还会改变字节数。块结点末尾的填充符与真实字符应靠长度或其他约定区分。

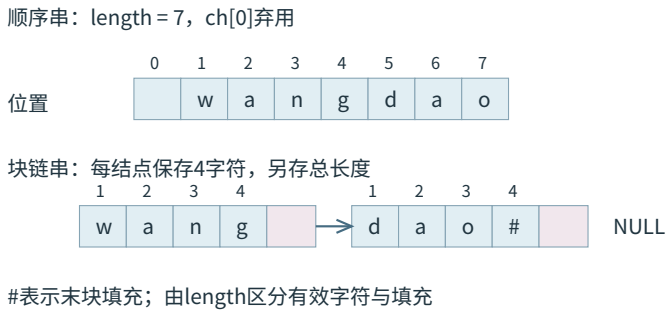


Figure 4.1: 顺序串的 1 起始字符位置与链块串的链接关系

4.3 基本操作怎样落实到代码

`SubString` 先检查范围，再从主串复制；失败时保留输出串原值。按固定数组模型，最多复制 `len` 个字符，时间 $O(len)$，除返回结果外只用常数个工作变量。

```
bool SubString(SString &Sub, SString S, int pos, int len) {
    if (pos < 1 || pos > S.length + 1 || len < 0 ||
        len > S.length - pos + 1)
        return false;
    for (int i = pos; i < pos + len; ++i)
        Sub.ch[i - pos + 1] = S.ch[i];
    Sub.length = len;
    return true;
}
int StrCompare(SString S, SString T) {
    for (int i = 1; i <= S.length && i <= T.length; ++i)
        if (S.ch[i] != T.ch[i])
            return (unsigned char)S.ch[i] -
                   (unsigned char)T.ch[i];
    return S.length - T.length;
}
```

比较的时间由第一次不同的位置决定，最坏 $O(\min(n,m))$。返回值只要求正、零、负正确；不能把“返回正值”误背成“必须返回 1”。转为 `unsigned char` 后比较，可以避免某些实现中 `char` 有符号导致高位编码被当作负数。

连接先检查总长，再分别复制两段。若接口允许输出与输入是同一个对象，需要按实现检查覆盖问题；下例按值接收定长顺序串，输入副本使 `Concat(S,S,T)`仍能读取原来的 S。

```
bool Concat(SString &T, SString S1, SString S2) {
    if (S1.length + S2.length >= MAXLEN) return false;
    for (int i = 1; i <= S1.length; ++i)
        T.ch[i] = S1.ch[i];
    for (int i = 1; i <= S2.length; ++i)
        T.ch[S1.length + i] = S2.ch[i];
    T.length = S1.length + S2.length;
    return true;
}
```

辨析·回溯到哪里

本轮起点是 i-j+1，失败后换下一起点 i-j+2，模式 j 恢复为 1。

若用字符串基本操作描述朴素定位，可以依次取长度 m 的子串，再用 `StrCompare` 比较。非空模式最多有 $n-m+1$ 个候选起点；模式长于主串时没有候选。下面改用字符位置直接实现，能更清楚地看到回溯和字符比较次数。

4.4 朴素匹配：失败后移到下一个起点

令 `i` 指主串当前比较字符，`j` 指模式当前比较字符，二者都从 1 开始。相等时同时递增；不等时，已经比较的模式前缀长为 $j-1$，本轮起点是 $i-j+1$，下一个起点是 $i-j+2$，因此执行 `i=i-j+2; j=1;`。

```
int Index(SString S, SString T) {
    if (T.length == 0) return 1;
    int i = 1, j = 1;
    while (i <= S.length && j <= T.length) {
        if (S.ch[i] == T.ch[j]) {
            ++i; ++j;
        } else {
            i = i - j + 2;
            j = 1;
        }
    }
    if (j > T.length) return i - T.length;
    return 0;
}
```

成功时，`j=m+1`，`i` 已越过匹配段，起点为 $i-m$。主串先耗尽而模式仍未完成时失败，返回 0。失败值 0 与合法位置 1 区分，不应改用另一个接口的 `-1`。

4.4.1 例：按课件主串逐次尝试

主串 `S=abaabaabcabaabc`，模式 `T=abaabc`。起点 1 处前 5 字符相同，第 6 字符 `a` 与 `c` 失配；起点 2 的 `b` 与模式首字符 `a` 失配；起点 3 先比较一个 `a`，下一字符 `a` 与 `b` 失配；起点 4 的 6 个字符全部相等，第一次成功位置为 4。

尝试起点	本轮字符比较次数	结果
1	6	第 6 对失配
2	1	第 1 对失配
3	2	第 2 对失配
4	6	全部匹配

合计 15 次字符比较。这个总数属于这一组数据，不能当作算法的一般复杂度。

最坏情形如主串前部有大量 `a`、模式为 `aaaab`：每个起点都在接近末尾处失败，时间为 $O((n-m+1)m)$，通常简记 $O(nm)$。若首起点就完全匹配，则只比较 m 次；讨论最好情况时须说明是否保证成功及是否把建表等准备工作计入。

辨析·位置与长度差 1

next[j] 是下一比较位置。j 前的最长边界长 k 时，next[j]=k+1。next[1]=0 另行定义。

4.5 KMP：利用已相等部分决定回退

设在位置 i, j 失配。此前有

$$S[i-j+1..i-1] = T[1..j-1].$$

这段主串已经被证明等于模式前缀。模式右移后仍能保留的部分，必须同时是 $T[1..j-1]$ 的真后缀和模式的真前缀。设最长相等真前后缀长为 k，那么前 k 字符无需再比较，下一次应将模式位置 $k+1$ 与同一个主串字符比较。

因此，对 $j > 1$ 定义

$$next[j] = 1 + \text{串} T[1..j-1] \text{ 的最长相等真前后缀长度},$$

并定义 $next[1] = 0$。`next[j]`保存的是**下次比较的位置**，不是直接保存前后缀长度。值 0 是首字符失配后的特殊状态：下一轮同时令 `i` 和 `j` 递增，主串进入下一字符，模式恢复到位置 1。

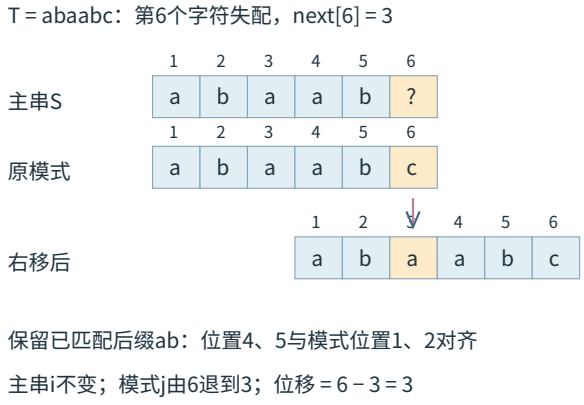


Figure 4.2: abaabc 在第 6 位置失配后，已匹配段的后缀 ab 与模式前缀 ab 对齐，j 由 6 退到 3

模式 `abaabc` 的普通 next 为：

位置 j	1	2	3	4	5	6
T.ch[j]	a	b	a	a	b	c
j 之前的串	空	a	ab	aba	abaa	abaab
最长相等真前后缀长	特殊	0	0	1	1	2
next[j]	0	1	1	2	2	3

例如 $j=6$ 前的串是 abaab，最长相等真前后缀为 ab，长 2，所以 next[6]=3。当第 6 字符失配时，只改变 j 到 3，i 不变；不是把模式移到主串位置 3。

辨析·主串没有回溯

失配时 i 保持不变，先改 j；只有字符相等或 j 退至 0 时才推进 i。

```
int Index_KMP(SString S, SString T, int next[]) {
    if (T.length == 0) return 1;
    int i = 1, j = 1;
    while (i <= S.length && j <= T.length) {
        if (j == 0 || S.ch[i] == T.ch[j]) {
            ++i;
            ++j;
        } else {
            j = next[j];
        }
    }
    if (j > T.length) return i - T.length;
    return 0;
}
```

短路求值使 j==0 时不访问 T.ch[0]。模式在逻辑上右移，主串指针单调递增；同一个主串字符仍可能与多个模式位置比较，所以不能将 KMP 解释成“每个主串字符恰好比较一次”。

4.5.1 例：同一主串上的 KMP 轨迹

仍取 S=abaabaabcabaabc、T=abaabc。先比较位置 $(1,1)$ 至 $(5,5)$ 成功，在 $(6,6)$ 失配。由 next[6]=3，下一次比较 $(6,3)$，主串位置 6 的 a 等于模式位置 3 的 a；随后比较 $(7,4)$、$(8,5)$、$(9,6)$ 都成功。此时 i=10,j=7，返回 $10-6=4$。

阶段	主串位置 i	模式位置 j	操作
已有 5 字符相等	6	6	a 与 c 失配
查 next	6	3	主串位置不变
继续比较	6、7、8、9	3、4、5、6	连续 4 对相等
完成	10	7	返回位置 4

本例普通 KMP 共比较 10 次字符，少于朴素的 15 次；判断 j==0 不算一对字符比较。另一课件主串 abaacaabcabaabc 会产生连续回退：在 $i=5,j=5$ 失配后 $j=2$，仍失配再退 $j=1$，再失配退 $j=0$；此后推进主串到 $i=6,j=1$。连续回退只查表，不让主串后退。

4.6 怎样在线性时间内求 next

求表也可看成模式串与自身前缀的匹配。设正在求 next[i+1]，已知 next[i]=j。若 j==0 或 T.ch[i]==T.ch[j]，当前相等前后缀可扩展一字符，故 next[i+1]=j+1；否则先尝试更短的候选边界，令 j=next[j]。

```
void get_next(SString T, int next[]) {
    if (T.length == 0) return;
    int i = 1, j = 0;
    next[1] = 0;
    while (i < T.length) {
        if (j == 0 || T.ch[i] == T.ch[j]) {
            ++i; ++j;
            next[i] = j;
        } else {
            j = next[j];
        }
    }
}
```

辨析·先切出 j 前的串

求 next[j] 只看 T[1..j-1]，当前位置字符不加入普通边界长度的计算。

这里的 `i` 是模式串内部的工作位置，与匹配函数的主串工作位置作用不同。`while(i<T.length)`保证递增后最多写到 `next[T.length]`。若使用另一种定义而把 `next` 记成前缀函数，表值和匹配代码就必须一起换；本章始终使用教材的下一比较位置定义。

4.6.1 三组课件手算练习

模式串	位置	普通 next
google	1、2、3、4、5、6	0、1、1、1、2、1
ababaa	1、2、3、4、5、6	0、1、1、2、3、4
aaaab	1、2、3、4、5	0、1、2、3、4

`google` 在位置 5 前的 `goog` 有长度 1 的相等真前后缀 `g`，所以 `next[5]=2`；位置 6 前的 `googl` 没有非空相等真前后缀，因此 `next[6]=1`。`ababaa` 在位置 6 前的 `ababa` 有最长边界 `aba`，所以 `next[6]=4`。`aaaab` 每个待考察前缀都是一串 `a`，其最长真边界只比它本身短 1。

课件主串 `googlogoogoogle`、模式 `google` 的匹配过程会反复利用这些边界，第一次成功位置为 10。手算时先列主串和模式的位置行，再记录每一次失配后的 `j`，比仅画模式移动的外形更容易核对。

4.7 nextval 怎样跳过必然失败的比较

若当前 `S.ch[i]`与 `T.ch[j]`已失配，而 `T.ch[next[j]]==T.ch[j]`，同一个主串字符与回退位置的字符也一定失配。可以继续沿失败链接，直接寻找字符不同的候选位置；把最终位置记为 `nextval[j]`。

由普通 next 求优化表时，先令 $nextval[1]=0$。为简化公式，记 $T[j]=T.ch[j]$，对 $j\ge 2$ 有：

$$nextval[j]=\begin{cases} next[j], & T[j]\neq T[next[j]],\\ nextval[next[j]], & T[j]=T[next[j]]. \end{cases}$$

对 $j\ge 2$，普通 `next[j]`至少为 1，所以该式不访问 `T.ch[0]`。`nextval` 使用已确认的失配信息，省去必定失败的比较，不改变匹配结果。

模式串	普通 next	优化 nextval
abaabc	0、1、1、2、2、3	0、1、0、2、1、3
ababaa	0、1、1、2、3、4	0、1、0、1、0、4
aaaab	0、1、2、3、4	0、0、0、0、4

辨析·两种表两种次数

aaaab 在 aaacaaaabdaaaab 中首次匹配位于 5；普通 next 比较 12 次，nextval 比较 9 次。

例如 `abaabc` 的 $j=3$ 字符为 `a`，而 `next[3]=1` 也指向 `a`，故 `nextval[3]=nextval[1]=0`；$j=5$ 是 `b`，普通回退到 2 仍为 `b`，故 `nextval[5]=nextval[2]=1`。

```
void get_nextval(SString T, int nextval[]) {
    if (T.length == 0) return;
    int i = 1, j = 0;
    nextval[1] = 0;
    while (i < T.length) {
        if (j == 0 || T.ch[i] == T.ch[j]) {
            ++i; ++j;
            if (T.ch[i] != T.ch[j])
                nextval[i] = j;
            else
                nextval[i] = nextval[j];
        } else {
            j = nextval[j];
        }
    }
}
```

将 `nextval` 作为 `Index_KMP` 的第三个实参即可使用优化表，匹配循环不变。普通 next 与优化 nextval 的具体比较次数可以不同，算题前先确认题目采用哪一张表。

4.7.1 例：重复字符使优化效果更明显

主串 `aaacaaaabdaaaab`，模式 `aaaab`。普通 next 在主串位置 4 的 `c` 处，会依次与模式位置 4、3、2、1 的 `a` 失配；优化表在位置 4 第一次失配后直接退到 0，跳过另外三次注定失败的比较。之后从主串位置 5 开始匹配，位置 5 至 9 恰为 `aaaab`，故返回 5。普通 next 共比较 12 次，nextval 共比较 9 次。

4.8 正确性、代价和容易混淆的边界

KMP 正确性的关键是：每一次跳过的对齐位置都被已经相等的前缀与后缀关系排除。模式右移后还能与旧匹配段重合的部分必须是一个边界；先尝试最长边界，失败后沿 next 链尝试更短边界，因此不遗漏仍可能成功的起点。

建表过程中，`i` 最多递增 $m-1$ 次；回退使 `j` 减小，而 `j` 的总增量受 `i` 的推进限制，故总工作 $O(m)$。匹配中，主串指针 `i` 最多递增 n 次，每次成功比较使 `j` 增加，每次失败回退使 `j` 减少；累积回退量仍受此前的推进约束，因此匹配扫描为 $O(n)$。总时间 $O(n+m)$，next 数组辅助空间 $O(m)$。只有“已给 next”的匹配阶段才可单独记为 $O(n)$。

问题	本章约定或结论	易混点
字符起始位置	1	与 C 数组常见 0 起始分别说明
匹配失败	返回 0	0 不是合法非空匹配起点
`next[j]`	下一次比较的模式位置	等于边界长度加 1，而非直接等于边界长度
`j==0`	下轮推进主串并恢复 j=1	不是读取第 0 个字符
字符比较次数	实际执行的字符相等判断	`j==0`、循环条件不是字符对比较
匹配成功后继续查找	取决于是否查全部、是否允许重叠	只找首次出现的接口可立即返回

固定数组按值传参会复制整个结构体；教材复杂度通常关注字符扫描和串长变化，容量视作预设上限。若要在可增长规模的软件实现中精确计算拷贝成本，应把接口实现成本一并计入。本章算法推导仍采用教材的字符比较模型。

辨析·失败值和合法位置

正文所有非空串位置从 1 起，失败返回 0；不要将另一种下标接口的返回值混入。

4.9 主动回忆与自查

以下为自拟检查题，串的位置和 next 均采用本章约定。

1. 空串和只含三个空格的串长度分别是多少？`abc` 是不是 `ac` 的主串？
2. 长度 7 的串取 `pos=6,len=2` 是否合法？`pos=8,len=0` 呢？
3. 字符位置为 1 起始时，朴素匹配在 $(i, j) = (8, 4)$ 失配，下一个起点和新的指针分别是什么？
4. 写出 `abaabc`、`ababaa`、`aaaab` 的 next 与 nextval，解释 `next[6]=3` 中 3 的含义。
5. KMP 在同一个主串字符上连续失配，会改变哪个指针？何时才推进主串？
6. 对 `S=abaabaabcabaabc,T=abaabc`，朴素与普通 KMP 第一次成功位置及字符比较次数分别是多少？
7. 对 `S=aaacaaaabdaaaab,T=aaaab`，普通 next 和优化 nextval 各比较多少次？
8. 空模式由接口约定返回 1 时，是否需要访问 next[1]？若模式长于主串，返回什么？
9. 只给匹配扫描的 $O(n)$，为何不足以说明整个 KMP 的准备与扫描代价？

4.9.1 核对答案

1. 分别为 0 和 3；`ac` 不是 `abc` 中的连续子串。2. 均合法；第二种取得串尾后的空串。3. 本轮起点为 5，下一个为 6，执行 `i=6,j=1`。4. next 和 nextval 分别见上表；3 表示下一比较位置，保留的是长 2 的边界 `ab`。5. 令 `j=next[j]`或 `j=nextval[j]`；字符相等或 j 退到 0 后推进主串。6. 都返回 4；朴素 15 次，普通 KMP10 次。7. 分别 12 次、9 次。8. 不需要建表或访问表；非空模式长于主串时返回 0。9. 还要把 $O(m)$ 建表时间和 $O(m)$ 表空间计入，总时间 $O(n+m)$。